COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

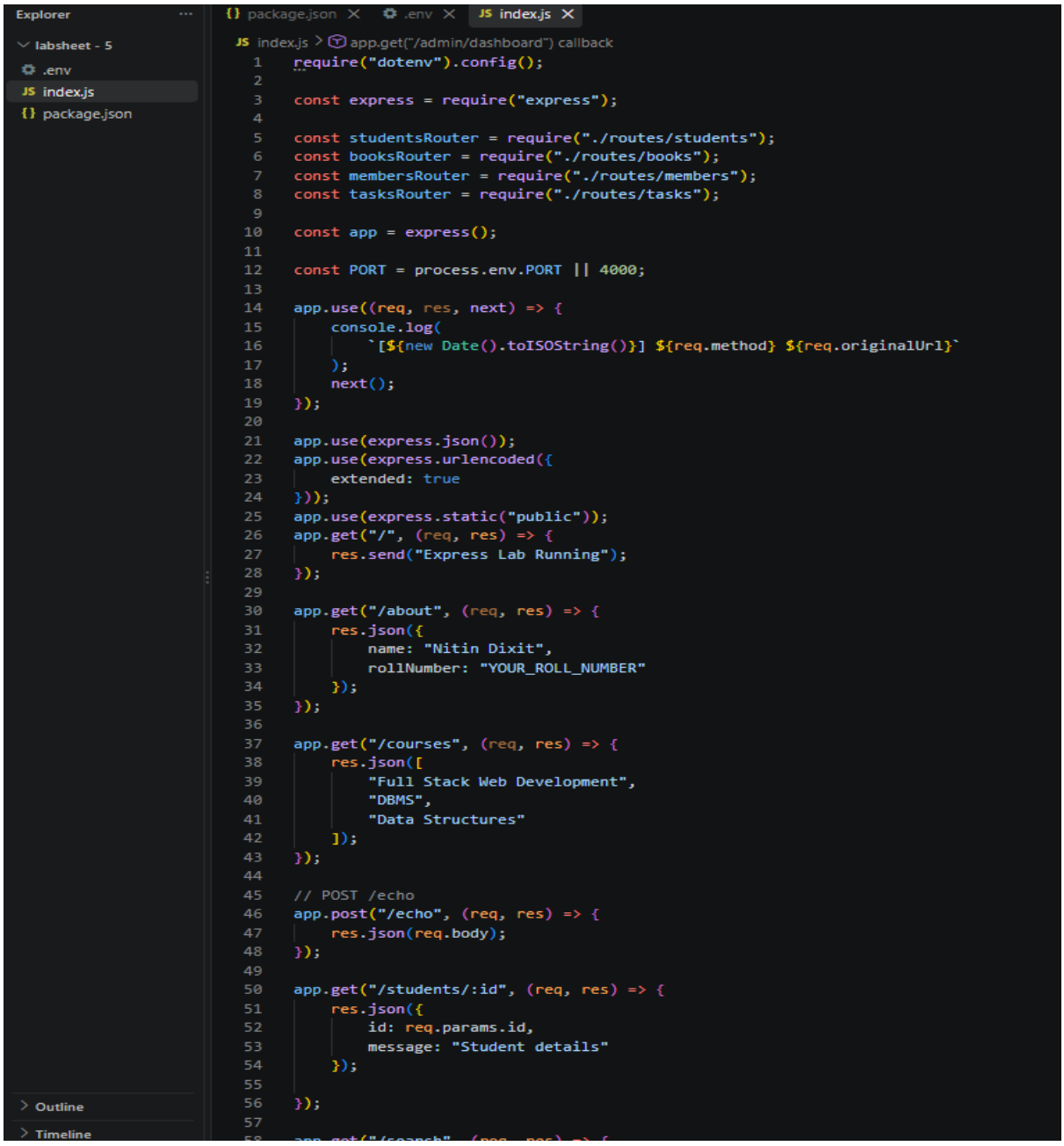
Course: Full Stack Web Development

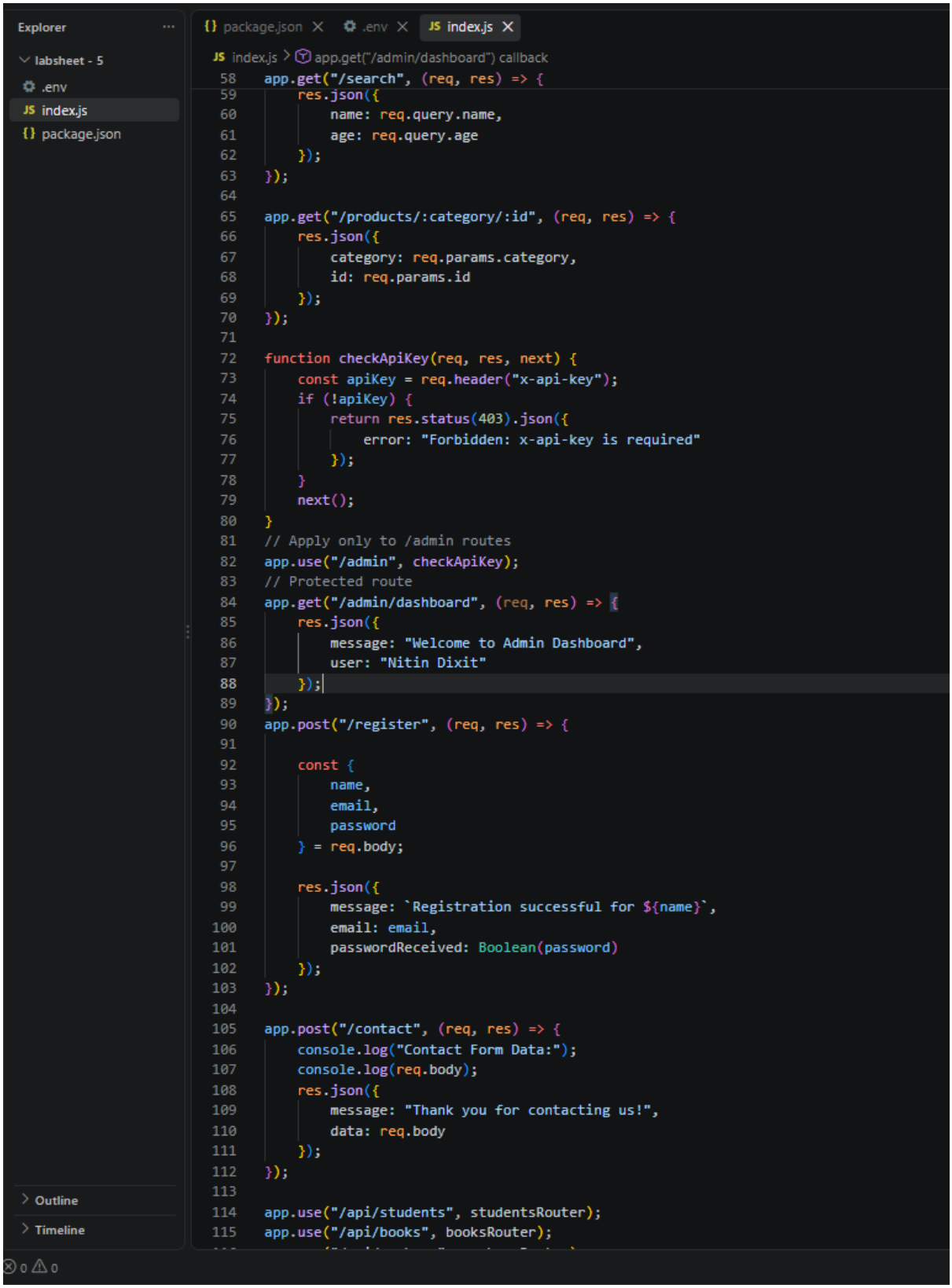
LAB SHEET 5— Express.js

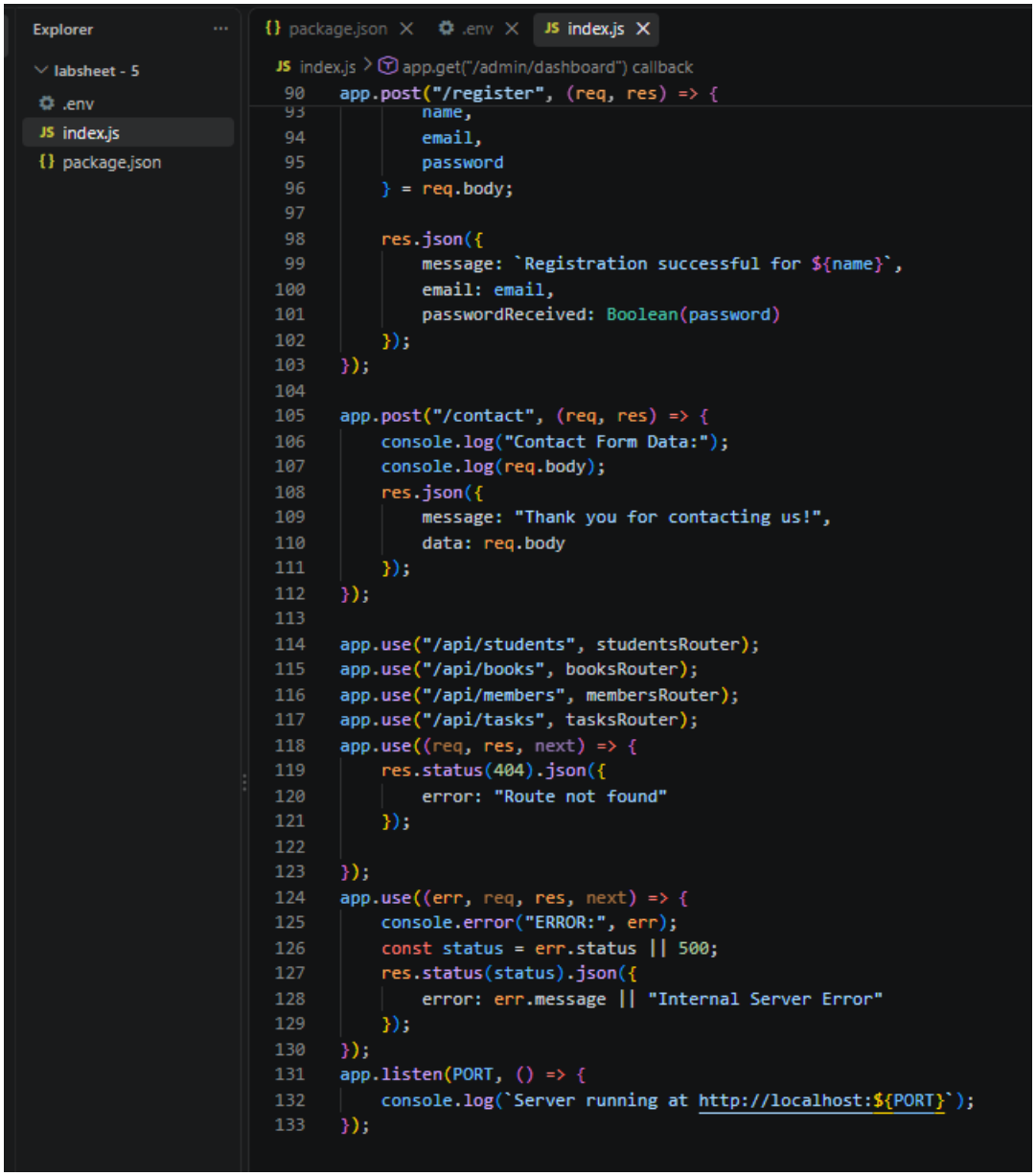
Name – Nitin Dixit

Roll no. – 38

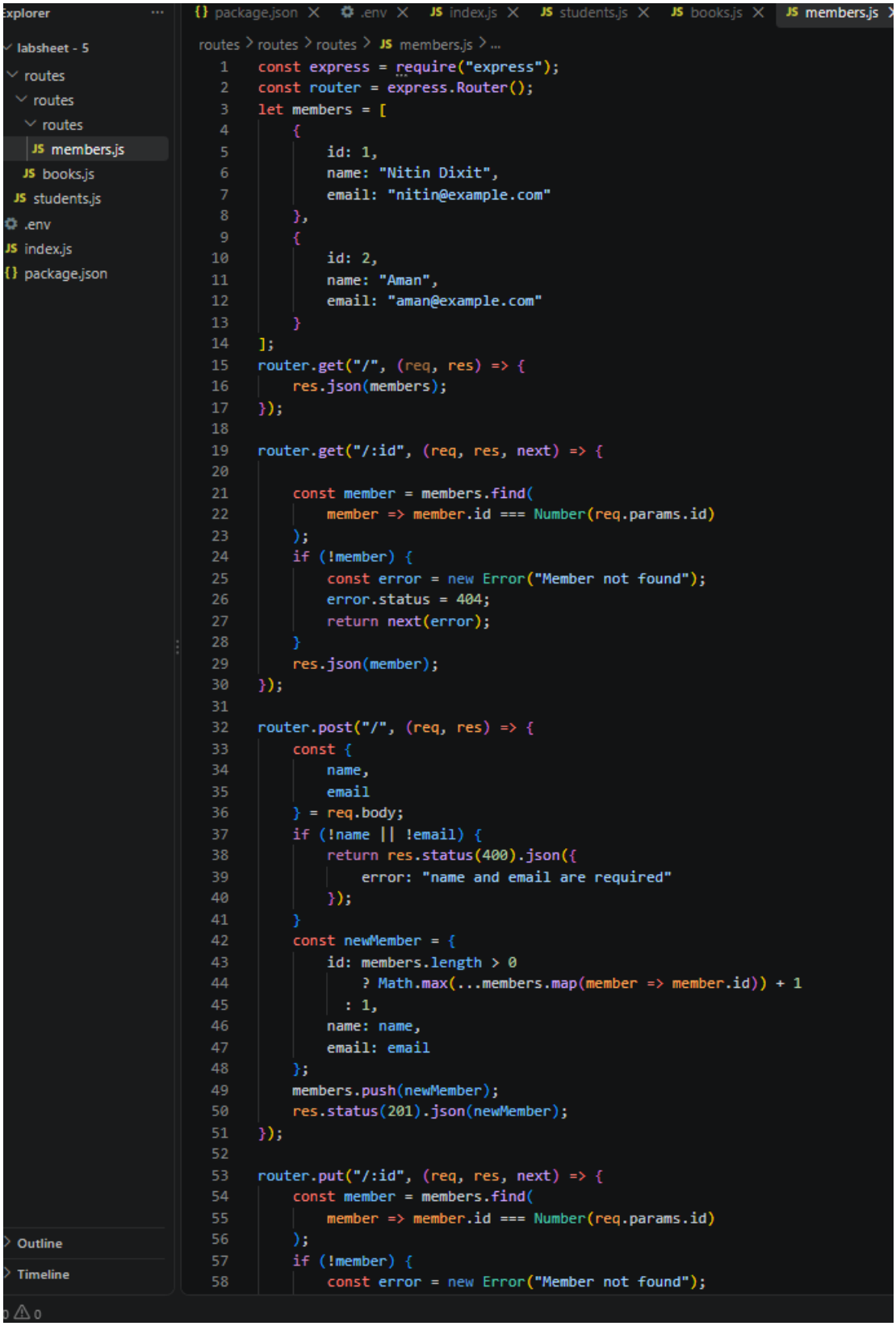
Section - B

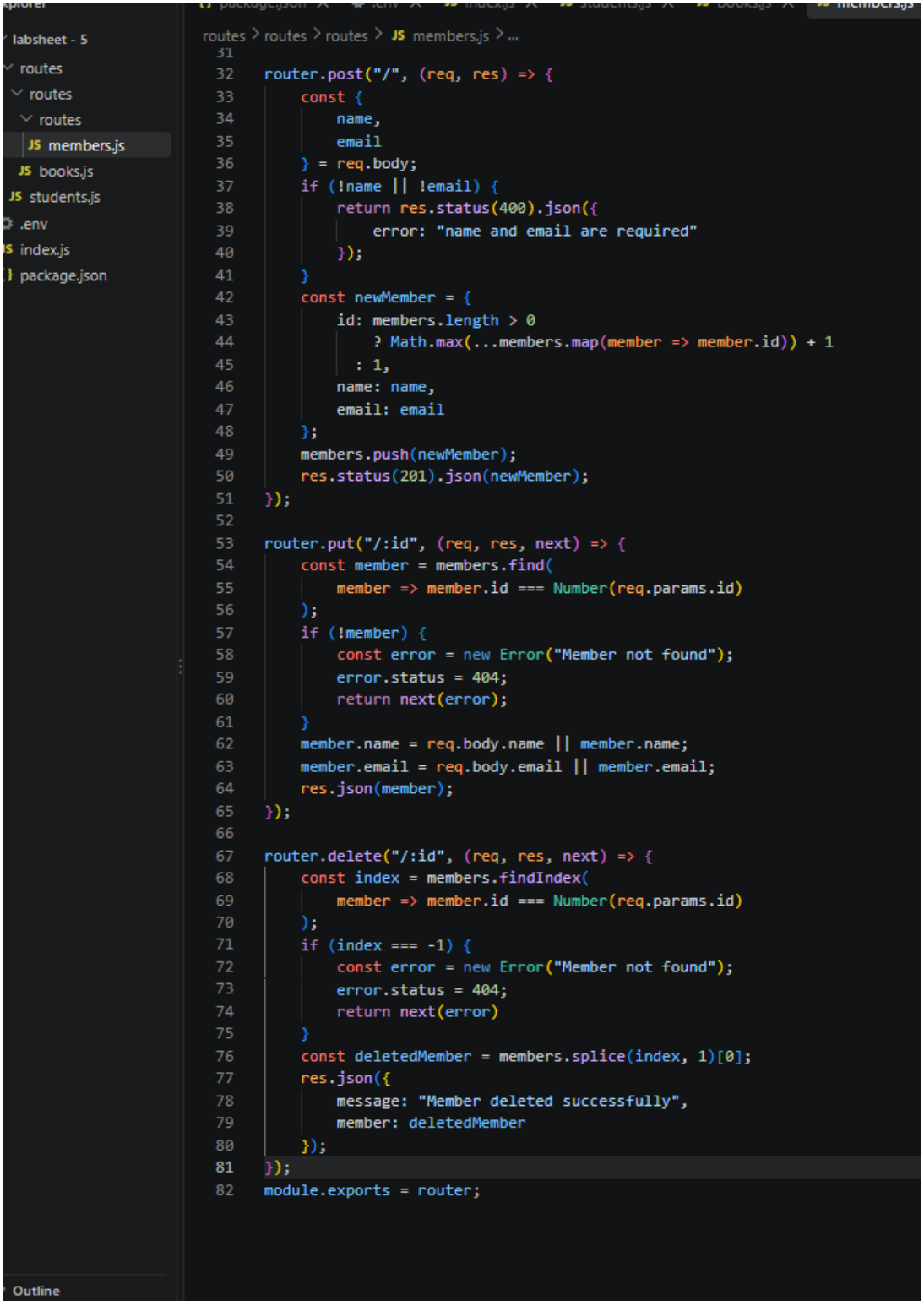


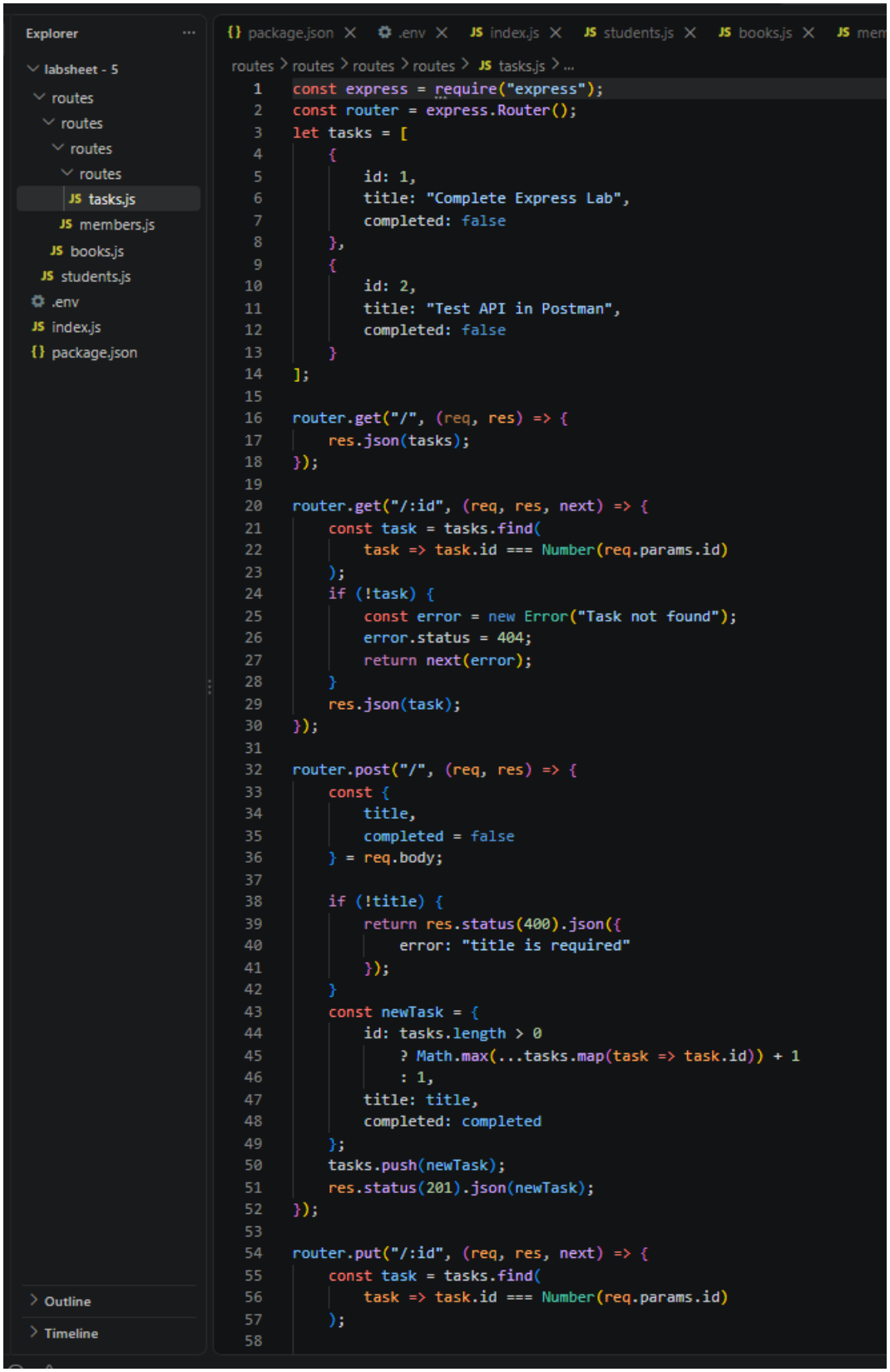

Explorer
labsheet - 5
.env
index.js
package.json
Outline
Timeline
package.json
.env
index.js
index.js > app.get("/admin/dashboard") callback
app.get("/search", (req, res) => {
res.json({
name: req.query.name,
age: req.query.age
});
});
app.get("/products/:category/:id", (req, res) => {
res.json({
category: req.params.category,
id: req.params.id
});
});
function checkApiKey(req, res, next) {
const apiKey = req.header("x-api-key");
if (!apiKey) {
return res.status(403).json({
error: "Forbidden: x-api-key is required"
});
}
next();
}
// Apply only to /admin routes
app.use("/admin", checkApiKey);
// Protected route
app.get("/admin/dashboard", (req, res) => {
res.json({
message: "Welcome to Admin Dashboard",
user: "Nitin Dixit"
});
});
app.post("/register", (req, res) => {
const {
name,
email,
password
} = req.body;
res.json({
message: `Registration successful for ${name}`,
email: email,
passwordReceived: Boolean(password)
});
});
app.post("/contact", (req, res) => {
console.log("Contact Form Data:");
console.log(req.body);
res.json({
message: "Thank you for contacting us!",
data: req.body
});
});
app.use("/api/students", studentsRouter);
app.use("/api/books", booksRouter);
0 0

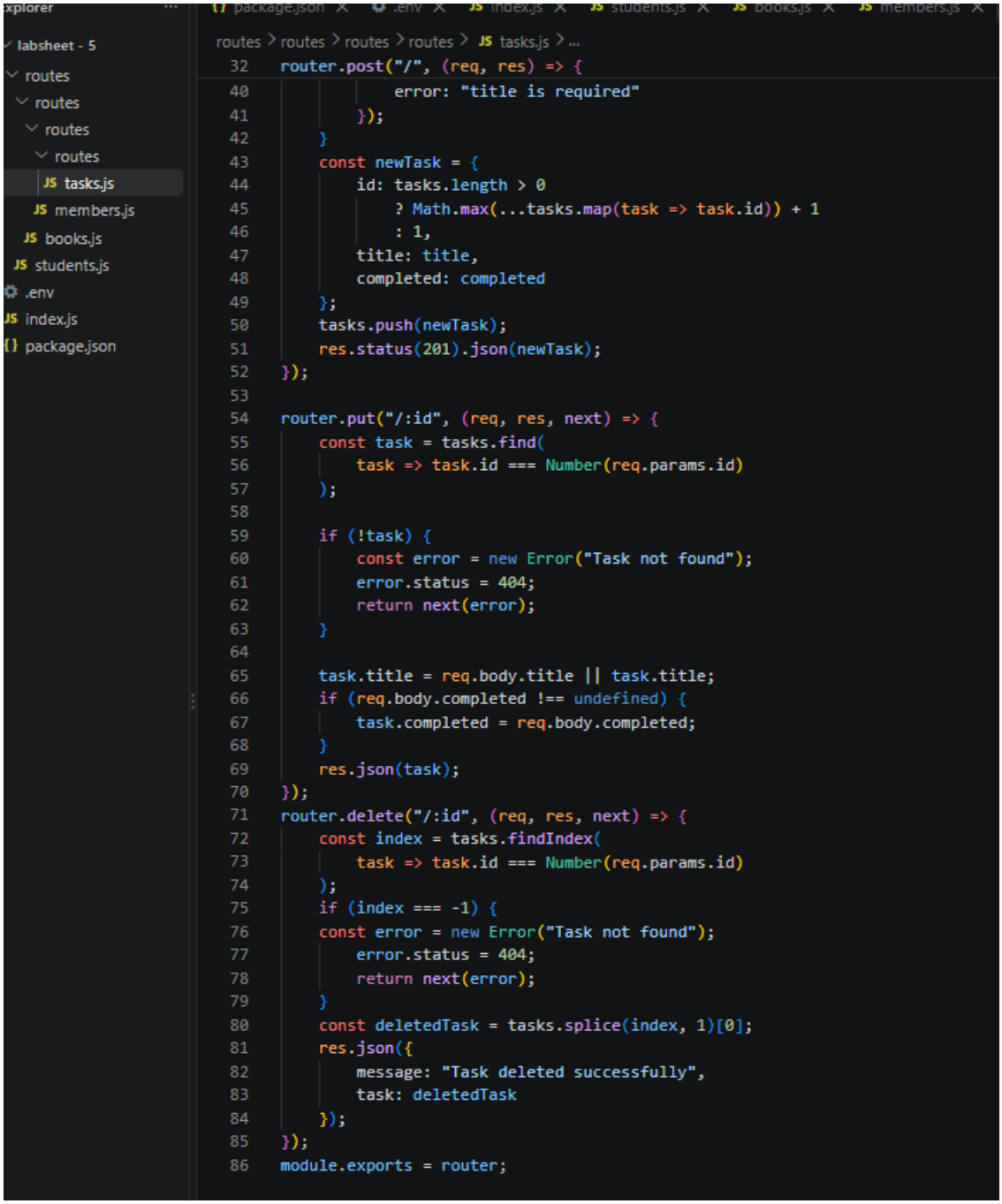

Explorer
labsheet - 5
.env
index.js
package.json
package.json
.env
index.js
index.js > app.get("/admin/dashboard") callback
app.post("/register", (req, res) => {
name,
email,
password
} = req.body;
res.json({
message: `Registration successful for ${name}`,
email: email,
passwordReceived: Boolean(password)
});
});
app.post("/contact", (req, res) => {
console.log("Contact Form Data:");
console.log(req.body);
res.json({
message: "Thank you for contacting us!",
data: req.body
});
});
app.use("/api/students", studentsRouter);
app.use("/api/books", booksRouter);
app.use("/api/members", membersRouter);
app.use("/api/tasks", tasksRouter);
app.use((req, res, next) => {
res.status(404).json({
error: "Route not found"
});
});
app.use((err, req, res, next) => {
console.error("ERROR:", err);
const status = err.status || 500;
res.status(status).json({
error: err.message || "Internal Server Error"
});
});
app.listen(PORT, () => {
console.log(`Server running at http://localhost:${PORT}`);
});

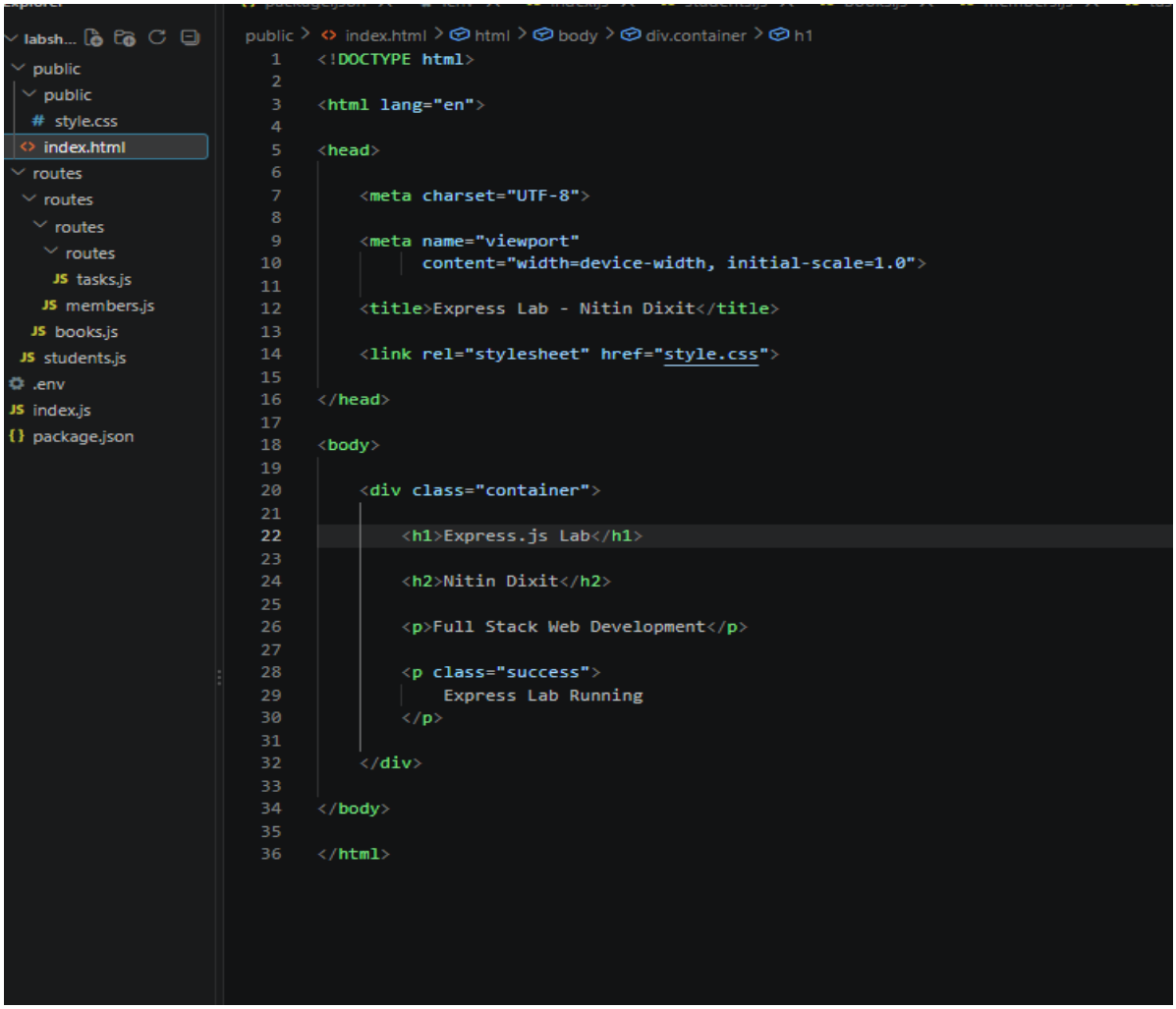

Explorer
labsheet - 5
routes
students.js
.env
index.js
package.json
package.json
.env
index.js
students.js
routes > students.js > ...
1 const express = require("express");
2
3 const router = express.Router();
4
5 const students = [
6 {
7 id: 1,
8 name: "Nitin Dixit",
9 course: "B.Tech CSE"
10 },
11 {
12 id: 2,
13 name: "Rahul",
14 course: "B.Tech CSE"
15 }
16];
17
18 // GET all students
19 router.get("/", (req, res) => {
20
21 res.json(students);
22
23 });
24
25 // GET student by ID
26 router.get("/:id", (req, res, next) => {
27
28 const student = students.find(
29 student => student.id === Number(req.params.id)
30);
31
32 if (!student) {
33
34 const error = new Error("Student not found");
35
36 error.status = 404;
37
38 return next(error);
39 }
40
41 res.json(student);
42
43 });
44
45 module.exports = router;


Explorer
labsheet - 5
routes
routes
routes
members.js
books.js
students.js
.env
index.js
package.json
Outline
Timeline
package.json
.env
index.js
students.js
books.js
members.js
routes > routes > routes > members.js > ...
const express = require("express");
const router = express.Router();
let members = [
{
id: 1,
name: "Nitin Dixit",
email: "nitin@example.com"
},
{
id: 2,
name: "Aman",
email: "aman@example.com"
}
];
router.get("/", (req, res) => {
res.json(members);
});
router.get("/:id", (req, res, next) => {
const member = members.find(
member => member.id === Number(req.params.id)
);
if (!member) {
const error = new Error("Member not found");
error.status = 404;
return next(error);
}
res.json(member);
});
router.post("/", (req, res) => {
const {
name,
email
} = req.body;
if (!name || !email) {
return res.status(400).json({
error: "name and email are required"
});
}
const newMember = {
id: members.length > 0
? Math.max(...members.map(member => member.id)) + 1
: 1,
name: name,
email: email
};
members.push(newMember);
res.status(201).json(newMember);
});
router.put("/:id", (req, res, next) => {
const member = members.find(
member => member.id === Number(req.params.id)
);
if (!member) {
const error = new Error("Member not found");


labsheet - 5
routes
routes
routes
members.js
books.js
students.js
.env
index.js
package.json
Outline
routes > routes > routes > JS members.js > ...
router.post("/", (req, res) => {
const {
name,
email
} = req.body;
if (!name || !email) {
return res.status(400).json({
error: "name and email are required"
});
}
const newMember = {
id: members.length > 0
? Math.max(...members.map(member => member.id)) + 1
: 1,
name: name,
email: email
};
members.push(newMember);
res.status(201).json(newMember);
});
router.put("/:id", (req, res, next) => {
const member = members.find(
member => member.id === Number(req.params.id)
);
if (!member) {
const error = new Error("Member not found");
error.status = 404;
return next(error);
}
member.name = req.body.name || member.name;
member.email = req.body.email || member.email;
res.json(member);
});
router.delete("/:id", (req, res, next) => {
const index = members.findIndex(
member => member.id === Number(req.params.id)
);
if (index === -1) {
const error = new Error("Member not found");
error.status = 404;
return next(error)
}
const deletedMember = members.splice(index, 1)[0];
res.json({
message: "Member deleted successfully",
member: deletedMember
});
});
module.exports = router;


Explorer
labsheet - 5
routes
routes
routes
routes
tasks.js
members.js
books.js
students.js
.env
index.js
package.json
Outline
Timeline
package.json
.env
index.js
students.js
books.js
routes > routes > routes > routes > tasks.js > ...
const express = require("express");
const router = express.Router();
let tasks = [
{
id: 1,
title: "Complete Express Lab",
completed: false
},
{
id: 2,
title: "Test API in Postman",
completed: false
}
];
router.get("/", (req, res) => {
res.json(tasks);
});
router.get("/:id", (req, res, next) => {
const task = tasks.find(
task => task.id === Number(req.params.id)
);
if (!task) {
const error = new Error("Task not found");
error.status = 404;
return next(error);
}
res.json(task);
});
router.post("/", (req, res) => {
const {
title,
completed = false
} = req.body;
if (!title) {
return res.status(400).json({
error: "title is required"
});
}
const newTask = {
id: tasks.length > 0
? Math.max(...tasks.map(task => task.id)) + 1
: 1,
title: title,
completed: completed
};
tasks.push(newTask);
res.status(201).json(newTask);
});
router.put("/:id", (req, res, next) => {
const task = tasks.find(
task => task.id === Number(req.params.id)
);

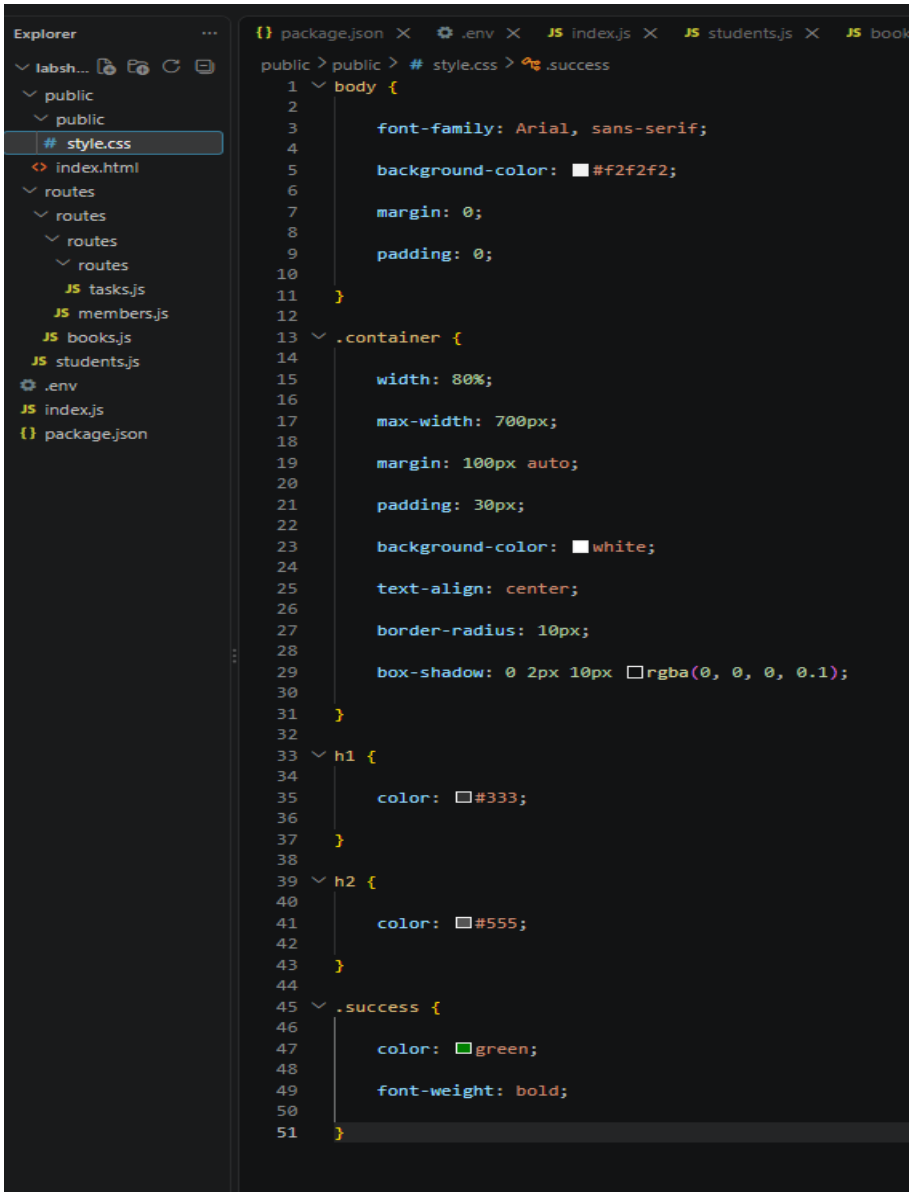

labsheet - 5
routes
routes
routes
routes
tasks.js
members.js
books.js
students.js
.env
index.js
package.json
routes > routes > routes > routes > tasks.js > ...
router.post("/", (req, res) => {
error: "title is required"
});
}
const newTask = {
id: tasks.length > 0
? Math.max(...tasks.map(task => task.id)) + 1
: 1,
title: title,
completed: completed
};
tasks.push(newTask);
res.status(201).json(newTask);
});
router.put("/:id", (req, res, next) => {
const task = tasks.find(
task => task.id === Number(req.params.id)
);
if (!task) {
const error = new Error("Task not found");
error.status = 404;
return next(error);
}
task.title = req.body.title || task.title;
if (req.body.completed !== undefined) {
task.completed = req.body.completed;
}
res.json(task);
});
router.delete("/:id", (req, res, next) => {
const index = tasks.findIndex(
task => task.id === Number(req.params.id)
);
if (index === -1) {
const error = new Error("Task not found");
error.status = 404;
return next(error);
}
const deletedTask = tasks.splice(index, 1)[0];
res.json({
message: "Task deleted successfully",
task: deletedTask
});
});
module.exports = router;

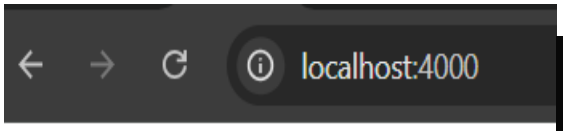

labsh...
public
public
style.css
index.html
routes
routes
routes
routes
tasks.js
members.js
books.js
students.js
.env
index.js
package.json
public > index.html > html > body > div.container > h1
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
<title>Express Lab - Nitin Dixit</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<h1>Express.js Lab</h1>
<h2>Nitin Dixit</h2>
Full Stack Web Development

Express Lab Running

</body>
</html>



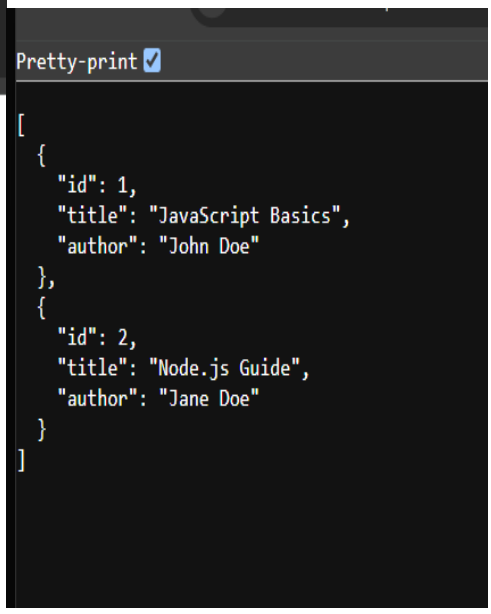


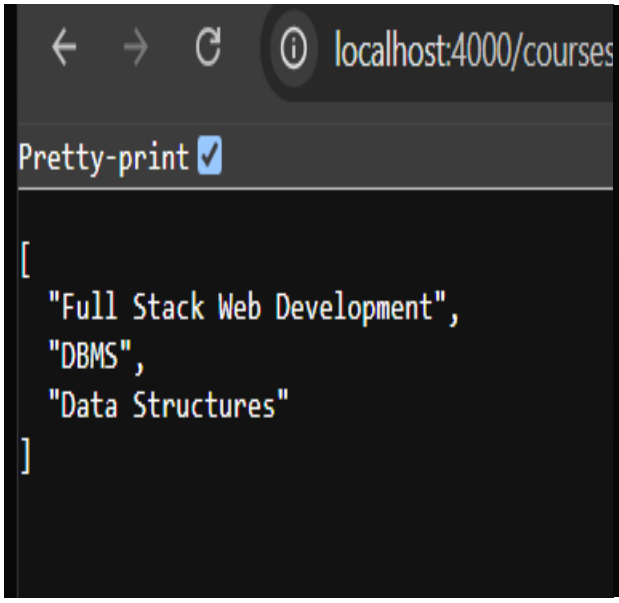
Express.js Lab

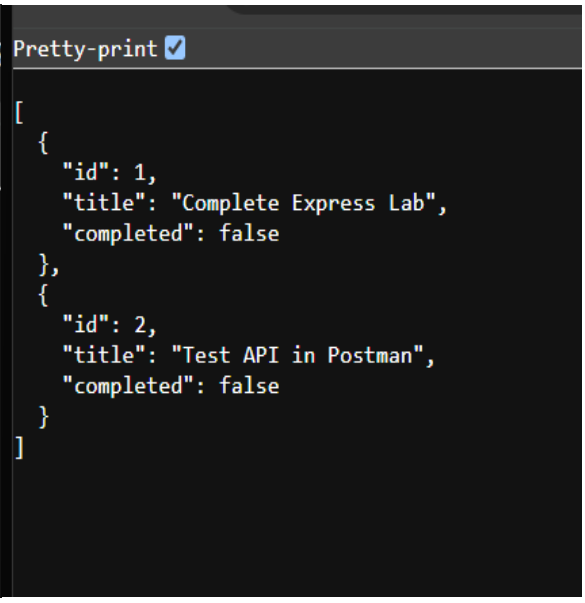
Nitin Dixit

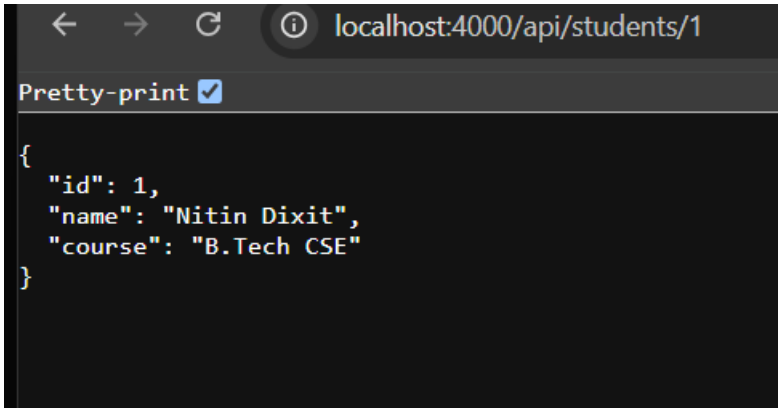
Full Stack Web Development

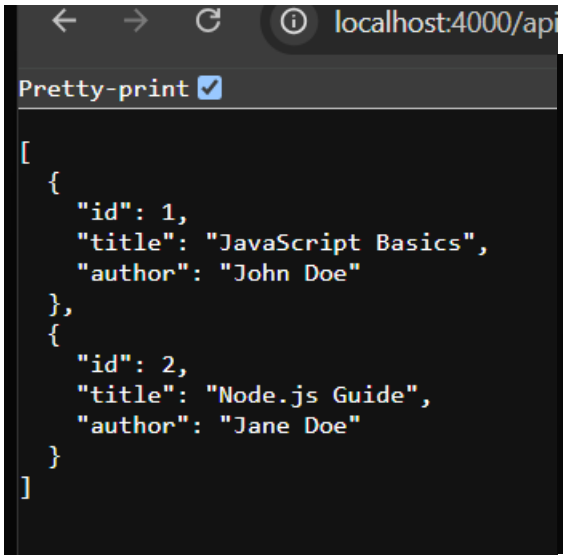
Express Lab Running

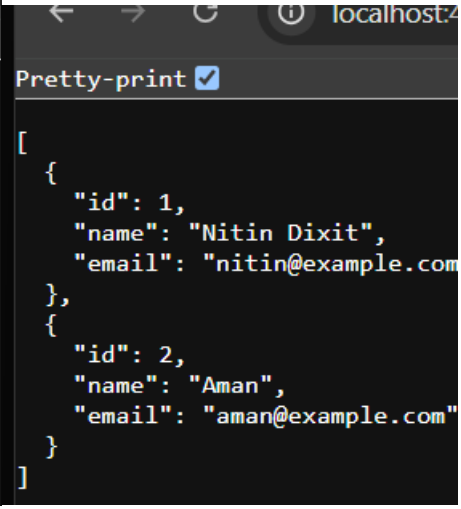



localhost:4000/courses
Pretty-print
[
"Full Stack Web Development",
"DBMS",
"Data Structures"
]


Pretty-print
[
{
"id": 1,
"title": "Complete Express Lab",
"completed": false
},
{
"id": 2,
"title": "Test API in Postman",
"completed": false
}
]


localhost:4000/api/students/1
Pretty-print
{
"id": 1,
"name": "Nitin Dixit",
"course": "B.Tech CSE"
}


localhost:4000/api
Pretty-print
[
{
"id": 1,
"title": "JavaScript Basics",
"author": "John Doe"
},
{
"id": 2,
"title": "Node.js Guide",
"author": "Jane Doe"
}
]


Pretty-print
[
{
"id": 1,
"name": "Nitin Dixit",
"email": "nitin@example.com
},
{
"id": 2,
"name": "Aman",
"email": "aman@example.com"
}
]